PHYS 3330 Final Project Proposal: 4-bit Digital Multiplier

Elijah Christie, Lev Kryvenko

October 26, 2026

1 Project Outline

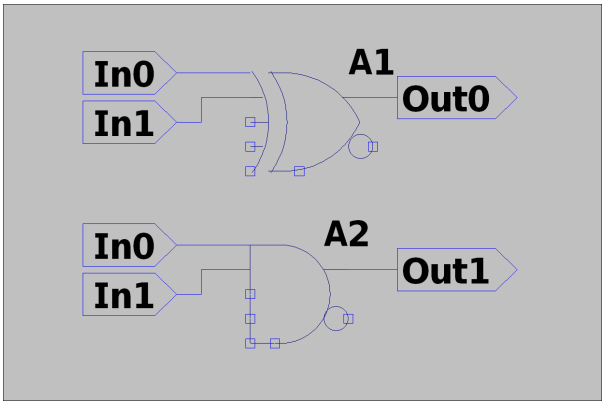
1.1 Description

On a high level, we plan to build a circuit that takes two 4-bit numbers and multiplies them using pure logic. This will be done using only elementary logic gates, there will be a total of 64 logic gates in this circuit. To interface with this circuit, we will use two dip switches for input. Then finally an Arduino will take the inputs and outputs and display each on 7-segment displays.

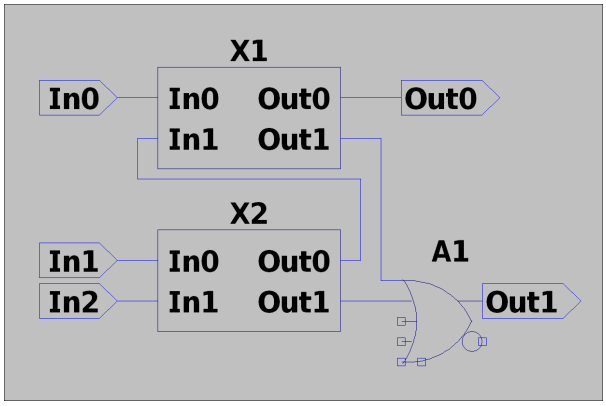
The circuit does not have an analog core as suggested in the project guidelines. But this idea of manually implementing a multiplier using only logic gates really captivated us, so we hope that the complexity of our circuit makes up for it being completely digital at its core. Also, our use of the Arduino is limited to the displays—we are not using it at any point for the core logical circuit.

1.2 Core Logical Multiplier Circuit

We have designed and validated a design for the internal circuit in LTSpice. While we will not use half-adder or full-adder gates in the actual circuit, we implemented them as parts in LTSpice for simplicity of designing the actual multiplier. Their schematics can be seen in Figure 1.



(a) Half-adder circuit. Takes two 1-bit inputs and sums them to a 2-bit output.



(b) Full-adder circuit. Takes three 1-bit inputs and sums them to a 2-bit output.

Figure 1: Self-contained adder circuits used for simplifying the schematic for the multiplier circuit.

In our actual circuit, we will not use any prefabricated half-adder or full-adder circuits. We will only use elementary logic gates for constructing the internal circuit. If we follow our current design, this will be done using only OR, XOR, and AND gates.

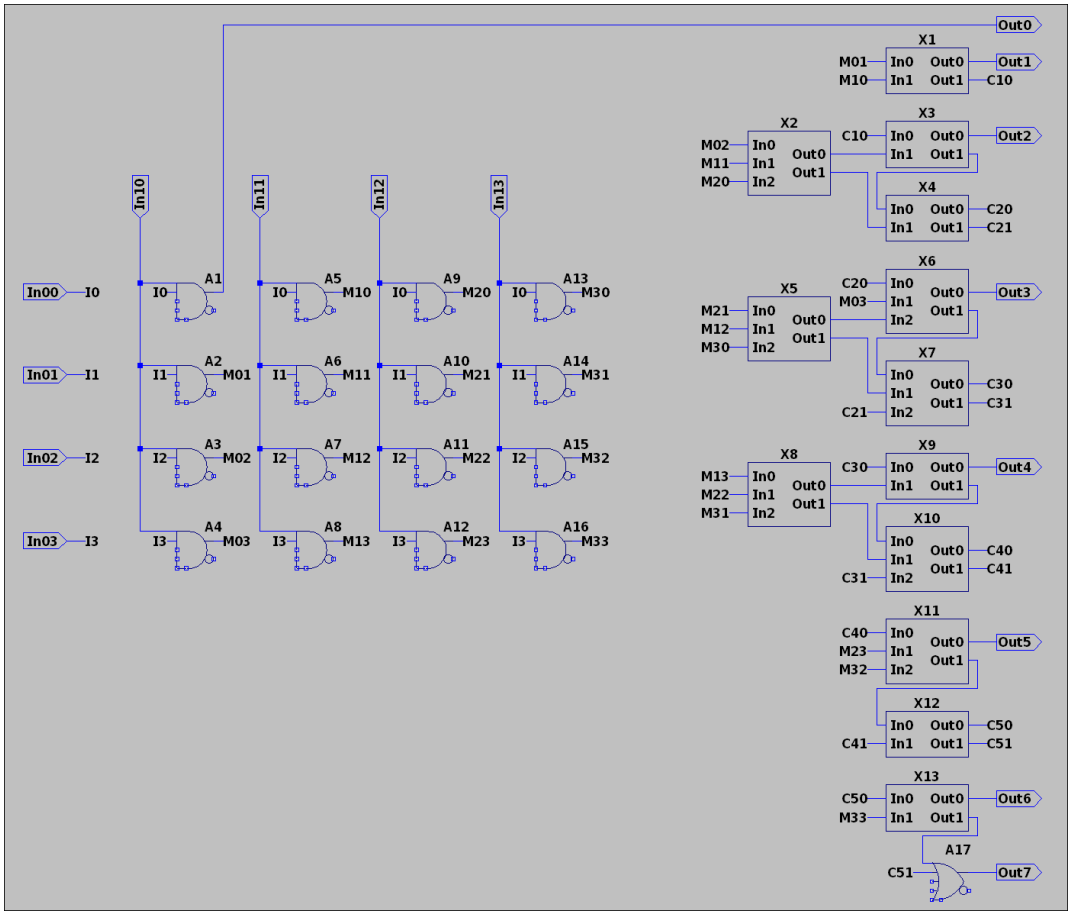


Figure 2: Multiplier circuit schematic designed using half- and full-adder gates. Takes two 4-bit inputs and multiplies them to an 8-bit output.

With this, our internal circuit's design can be seen in Figure 2. The first part of this circuit consists of a matrix of AND gates. In essence, what it's doing is that it's taking each digit of the input 1 and multiplying by input 0. So the first column is $I_{0*} \times I_{10}$, the second is $I_{0*} \times I_{11}$, and so on. The results from this multiplication are saved into MXX.

The next section of the circuit then adds these four numbers. This starts out simple, since M00 maps directly to Out0 (we had to connect it directly otherwise LTSpice wouldn't run our simulation). While the mini-circuit for each subsequent output is unique, they all share the same general design of (1) adding all the corresponding digits using a mix of adders, (2) determining what the OutX should be, and (3) what to carry over. The carry CXX is itself a binary number, most often 2-bit. The final output only has an OR gate because X13-Out1 and C51 are never high simultaneously, but they can be individually.

While the final result does require 8 bits to store, the largest it can be is only $15 \times 15 = 225$.

1.3 Interfacing with the Circuit

To interface with the circuit, we will install two dip switches and seven digits of 7-segment displays. The dip switches are dials which output binary numbers from 0 to 15 and will act as our input. Given that both our inputs and outputs are binary numbers, we decided that it will be most straightforward to use a single Arduino to read the binary numbers and output each to 7-segment

displays. We explored the option to use binary-to-BCD converters and related ICs, but ultimately using a single Arduino seemed easier and more cost-efficient.[1] The Arduino will read each of the inputs, as well as the output, and using a script (which we will write) convert the binary data to seven digits of 7-segment displays.

2 Parts

2.1 Core Circuit

Counting the gates in the internal circuit, we find that we will utilize 8 OR gates, 20 XOR gates, and 36 AND gates. If we were to use quad-chips like those we have already been using in labs, this would be at least 2 OR chips, 5 XOR chips, and 9 AND chips.[2] If we'd need to purchase all these gates ourselves, we could pick up SN74HC32N, SN74HC86N, and SN74HC08N.

2.2 Interface

The first part we'd need is the dip switch. After browsing DigiKey for a bit, we found that PT65503 looks to suit our needs well—we would get two of these. Then we'd need our 7-segment displays. Since we've got two inputs 0-15 and one output 0-225, we want two 2-digit displays and one 3-digit display. It seems like LTD-2701JD and LTC-4724JR can do the job well. To power these displays, we'd also use some of the resistors available in the lab.

We have $2 \times 4 + 8 = 16$ bits of binary and $7 \times 7 = 49$ LED segments. Technically, due to the limited digits displayed in the largest position on each display (blank or one for 2-digit, blank, one, or two for 3-digit), we can exclude at least 11 segments. With this, we need an Arduino with at least $16 + 38 = 54$ pins. From looking it up online, an Arduino Mega should be sufficient. For example, A000067.

2.3 Bill of Materials

If we were to purchase all the components new, you can see the total costs in Figure 3. All items that we've picked out are available directly from DigiKey.

Part	Description	Price	Count	Total Cost
SN74HC32N	OR Quad-Gate (x10)	\$5.27	1	\$5.27
SN74HC86N	XOR Quad-Gate (x10)	\$8.15	1	\$8.15
SN74HC08N	AND Quad-Gate (x10)	\$5.62	1	\$5.62
PT65503	Hexadecimal Dip Switch	\$10.42	2	\$20.84
LTD-2701JD	2-Digit 7-Segment Display	\$2.54	2	\$5.08
LTC-4724JR	3-Digit 7-Segment Display	\$3.55	1	\$3.55
A000067	Arduino Mega2560	\$49.90	1	\$49.90

Figure 3: Bill of materials. Most items listed will not have spares. The subtotal is \$98.41, or \$113.24 with shipping and tax.

[1] Most of the ICs are only sold bulk, costing as much as \$300, and have very long delivery times. Since these are all specialized components anyway, we felt that there's no objective reason to use them over an Arduino.

[2] It might be more convenient to put separation between some of these chips, hence why "at least".